UNIVERSITY OF MINES AND TECHNOLOGY, TARKWA

FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT REPORT ENTITLED

DESIGN AND IMPLEMENTATION OF A KNOWLEDGE GRAPH BASED MULTI-AGENT ORCHESTRATION FRAMEWORK

BY

KASSER-TEE ATHANASIUS YINTEMANN, EMILY WOBI

EDEM SOKATSI, NUGAH SELASE MARK

NOAH OTINKORANG BOYE

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

PROJECT SUPERVISOR

.....

Mr. Frederick E. Broni

Tarkwa, Ghana

March, 2026

DECLARATION

We, ..., declare that this project work is our own original work. It is being submitted for the degree of Bachelor of Science in Computer Science and Engineering at the University of Mines and Technology (UMaT), Tarkwa. It has not been submitted, either in whole or in part, for any degree or examination at any other university.

Signature of Student: _____

[Sokatsi Edem] (FOE.41.008.142.22)

Signature of Student: _____

[Nugah Selase Mark] (FOE.41.008.111.22)

Signature of Student: _____

[Wobi Emily Wepiah] (FOE.41.008.152.22)

Signature of Student: _____

[Kasser-Tee Athanasius Yintemann] (FOE.41.008.087.22)

Signature of Student: _____

Noah Otinkorang Boye (FOE.41.008.063.22)

_____ day of _____ (year) _____

ABSTRACT

Current LLM Based Multi-Agent system.....

DEDICATION

I dedicate....

ACKNOWLEDGMENTS

I attribute....

TABLE OF CONTENTS

DECLARATION **i**

ABSTRACT **ii**

DEDICATION **iii**

ACKNOWLEDGMENTS **iv**

TABLE OF CONTENTS **v**

LIST OF FIGURES **vii**

LIST OF TABLES **viii**

LIST OF ABBREVIATIONS **ix**

CHAPTER 1 INTRODUCTION **1**

1.1 Background Study 1

1.2 Problem Statement 3

1.3 Aim of the Study 4

1.4 Objectives 5

1.5 Methods Used 5

1.6 Tools and Facilities 6

1.6.1 Hardware 6

1.6.2 Software 6

1.7 Scope 6

1.8 Work Organization 6

CHAPTER 2 LITERATURE REVIEW **8**

2.1 Introduction to Concepts 10
2.1.1 LLM-Based Agents 10
2.1.2 Multi-Agent Systems 12

CHAPTER 3 CHAPTER THREE TITLE GOES HERE 14
3.1 FIRST SECTION TITLE GOES HERE 14
3.1.1 Your subsection 14

CHAPTER 4 CHAPTER TITLE HERE 15
4.1 FIRST SECTION TITLE GOES HERE 15
4.1.1 Your subsection 15

CHAPTER 5 CONCLUSION AND RECOMMENDATION 16
5.1 Conclusion 16
5.2 Recommendations 16

REFERENCES 17

LIST OF FIGURES

Figure	Title	Page
1.1	Recent trends in LLM-based multi-agent research for problem solving and world simulation are categorized and tracked by paper counts every three months, with each leaf node showing the number of papers in that category adapted from Guo *et al.* (2024)	1
2.1	The modular architecture of an LLM-based Agent, illustrating the interaction between reasoning core, memory, and tools.	10
2.2	Illustration of a Multi-Agent System (MAS) architecture, reproduced from Yang *et al.* (2024)	12

LIST OF TABLES

Table **Title** **Page**

LIST OF ABBREVIATIONS

DAG	Direct Acyclic Graph
KG	Knowledge Graph
AI	Artificial Intelligence
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
MAS	Multi-Agent System
API	Application Programming Interface
TOT	Tree of Thoughts
CoT	Chain of Thought
ReAct	Reasoning and Acting

CHAPTER 1

INTRODUCTION

1.1 Background Study

Large Language Model (LLM)-based multi-agent systems, where several LLM-powered agents collaborate to accomplish tasks that would otherwise be difficult for a single agent, have demonstrated significant potential in automating complex procedures within business workflows. These systems leverage the reasoning, planning, and language understanding capabilities of LLMs while distributing responsibilities across specialized agents (Weng, 2023). As a result, multi-agent architectures have emerged as a promising paradigm for building intelligent systems capable of handling multi-step and domain-specific tasks.

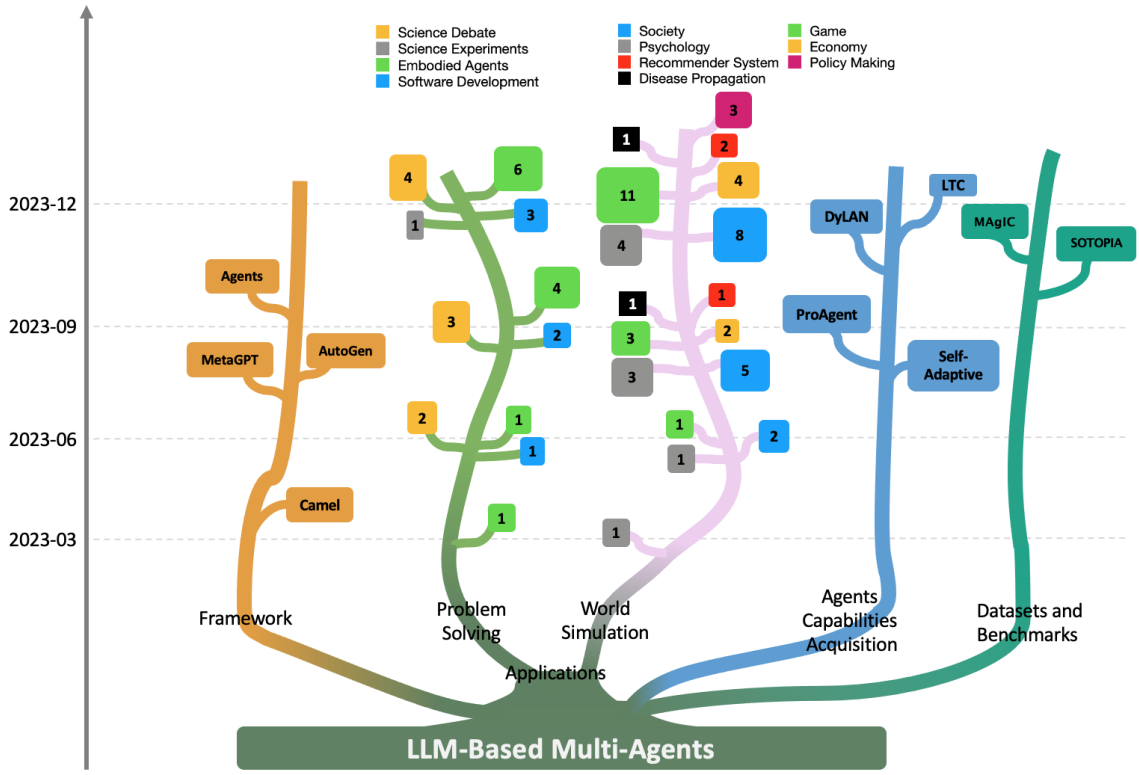

Figure 1.1 Recent trends in LLM-based multi-agent research for problem solving and world simulation are categorized and tracked by paper counts every three months, with each leaf node showing the number of papers in that category adapted from Guo *et al.* (2024)

Recent research indicates a growing interest in agentic AI frameworks such as LangGraph, CrewAI, and AutoGen. These frameworks provide architectural foundations and tooling for

designing, orchestrating, and deploying intelligent agent systems (Derouiche *et al.*, 2025). They enable developers to define agents with specialized roles, establish communication mechanisms between agents, and coordinate task execution across multiple components. According toTran *et al.* (2025), current multi-agent collaboration mechanisms can be categorized across several dimensions. Collaboration types include cooperation, competition, and coopetition, describing how agents interact to achieve their objectives. Collaboration strategies may be rule-based, role-based, or model-based, depending on how agent behavior is defined and coordinated. Communication structures may follow centralized, decentralized, or hierarchical patterns, while orchestration architectures may be static or dynamic depending on the nature of the task and domain requirements. These design principles collectively describe the operational foundations of modern multi-agent frameworks such as LangGraph, CrewAI, and AutoGen.

In practice, frameworks such as LangGraph and AutoGen often recommend the use of Directed Acyclic Graphs (DAGs) to model workflows that require strict sequencing and deterministic execution. This approach is particularly common in enterprise and business workflows, where the order of task execution must be explicitly defined to reduce uncertainty arising from the probabilistic nature of LLMs. However, DAG-based workflow definitions require developers to manually encode the sequence of tasks in advance. This introduces limitations in scenarios where workflows must adapt dynamically based on task context, domain knowledge, or intermediate outcomes.

Knowledge graphs provide a complementary approach to addressing this challenge. Knowledge graphs represent information using semantic relationships between entities, enabling datasets to be connected through their underlying meaning rather than purely structural links Hogan *et al.* (2021). By representing entities, relationships, and constraints within a domain, knowledge graphs offer a structured and extensible way to model domain knowledge. Recent research Nizar *et al.* (2025), Sanmartin (2024), and Kau *et al.* (2024) etc. have demonstrated the potential of integrating knowledge graphs with LLM-based systems. For example, knowledge graphs have been used to enhance retrieval-augmented generation, such as in KG-RAG (Sanmartin, 2024), to improve tool and agent retrieval mechanisms (Nizar *et al.*, 2025), and to ground LLM reasoning in structured knowledge representations (Amayuelas *et al.*, 2025). Other applications include knowledge injection, semantic reasoning, and improving

contextual understanding in LLM systems (Kau *et al.*, 2024).

Building on these developments, this project aims to design and implement a knowledge graph based multi-agent orchestration framework that leverages knowledge graphs to dynamically infer task sequencing within a workflow. Unlike traditional approaches that require explicit workflow definitions, the proposed framework relies on the knowledge graph to provide domain grounding, guide task decomposition, and facilitate tool and agent delegation. By leveraging semantic relationships between tasks, tools, and domain entities, the system can dynamically determine appropriate execution paths for a given workflow.

1.2 Problem Statement

Enterprise and business workflows typically require clearly defined task sequencing and explicit role delegation in order to ensure reliable and predictable execution Elementum Team (2026). In multi-agent systems, this requirement is commonly addressed using structured workflow representations such as Directed Acyclic Graphs (DAGs), where nodes represent tasks or agents and edges define dependencies that determine execution order Nath (2025).

Modern agent orchestration frameworks such as LangGraph often adopt DAG-based workflow structures to model task dependencies and ensure deterministic execution paths. DAGs enable systems to enforce task ordering, avoid cyclic dependencies, and manage complex workflows by representing tasks as nodes connected through directed edges LangChain Inc. (2024). This structure has been widely adopted in workflow orchestration and pipeline management systems due to its simplicity and reliability in representing sequential processes (Shankar, 2025).

While DAG-based orchestration is well-suited to structured workflows, it may present challenges in larger systems supporting many variations of agent–tool combinations. For example, one might anticipate the need to define and maintain separate DAGs for different workflows, which could introduce additional complexity. The process of designing, updating, and managing these DAG structures becomes increasingly complex and difficult to scale as the number of workflows increases. Furthermore, in cases where new agents or tools need to be introduced into the system, existing DAGs often need to be reconstructed or modified to incorporate these new components. This requirement increases the maintenance burden and reduces system flexibility, particularly in dynamic environments where agents and tools

evolve over time. This limitation creates a critical gap in current multi-agent orchestration approaches: workflow structures must be explicitly defined rather than dynamically inferred. As a result, systems lack the ability to automatically determine appropriate task sequencing based on contextual knowledge of available agents, tools, and domain constraints.

Consequently, an important research question arises: Can workflow structures be automatically generated instead of explicitly defined? Furthermore, how much contextual or domain knowledge is required for a system to infer appropriate task sequencing and agent delegation dynamically?

This research addresses this gap by proposing a Knowledge Graph (KG)-based multi-agent orchestration framework that automatically infers workflows by leveraging semantic relationships within a knowledge graph. Instead of relying on predefined DAGs, the proposed framework models relationships between tasks, agents, and tools within a knowledge graph and determines appropriate execution sequences by traversing this structured representation of domain knowledge. In addition, the proposed solution supports dynamic agent discovery, allowing new agents or tools to be incorporated into the system without requiring reconstruction of existing workflows. By properly linking newly introduced agents or tools to the knowledge graph through defined semantic relationships, they immediately become part of the orchestration ecosystem and can be discovered and utilized during workflow inference.

Through this approach, the framework aims to provide a scalable, adaptive, and extensible orchestration mechanism for multi-agent systems, reducing the need for manually defined workflows while enabling flexible integration of new agents and tools.

1.3 Aim of the Study

The aim of this research is to propose a knowledge graph–based multi-agent orchestration framework that dynamically infers workflow structures in LLM-based multi-agent systems. The framework leverages semantic relationships within a knowledge graph to guide task sequencing, agent selection, and tool delegation without requiring explicitly predefined workflows such as Directed Acyclic Graphs (DAGs).

1.4 Objectives

- To design a knowledge graph–based multi-agent orchestration framework, including task representation, agent modeling, and a traversal strategy for dynamic task sequencing and agent/tool selection.
- To develop an orchestrator agent to coordinate agents and manage workflow execution within the proposed framework.
- To implement and evaluate a prototype application to assess the framework's effectiveness in dynamically generating workflows and supporting multi-agent interactions.

1.5 Methods Used

This research follows a Systems-Oriented Research Methodology, focusing on the architectural design, implementation, and empirical evaluation of a novel orchestration engine designed to dynamically infer workflows in LLM-based multi-agent systems using Knowledge Graphs.

- A conceptual architecture for the knowledge graph–based orchestration framework is designed, defining the structure for task entry, agent representation, task delegation, and task completion.
- A knowledge graph schema is developed to represent the relationships between tasks, agents, and tools. This structure provides the semantic foundation required for workflow inference.
- An orchestrator agent is implemented to coordinate task execution, interact with the knowledge graph, and delegate tasks to appropriate agents or tools.
- A traversal mechanism is designed to enable the system to navigate relationships within the knowledge graph in order to infer task sequencing and identify suitable agents or tools for execution.
- A prototype system implementing the proposed framework is developed to demonstrate its functionality and validate the design.

- The framework is evaluated using selected task scenarios to assess its ability to dynamically generate workflows, coordinate agents, and integrate additional agents or tools within the system.

1.6 Tools and Facilities

1.6.1 Hardware

- MacBook Air M2, 16GB RAM / MacBook Air M3, 16GB RAM

1.6.2 Software

- Foundational LLM model via API or local deployment
- Knowledge Graph database (Neo4j)
- Python 3.10+
- Vector database (chromadb)

1.7 Scope

This research focuses on the design and implementation of a knowledge graph–based multi-agent orchestration framework for LLM-powered agent systems. The study covers the design of the framework architecture, development of an orchestrator agent, modeling of a knowledge graph to represent tasks, agents, and tools, and the implementation of a traversal strategy to enable dynamic workflow inference. A prototype application will be developed to demonstrate the functionality of the framework and evaluate its ability to coordinate agents and generate workflows dynamically. The scope of this research does not include the training or evaluation of large language models, as the study focuses on the orchestration framework rather than improvements to the underlying LLM models.

1.8 Work Organization

This project work is organized into five chapters. Chapter One deals with the introduction establishing context and goals. Chapter Two focuses on literature review, analyzing current approaches to Multi-Agent Orchestration and application of KG to ground LLM reasoning.

Chapter Three details the methodology and KG Orchestration Framework design and implementation. Chapter Four system testing and discussion with comprehensive evaluation results. Chapter Five provides conclusion and recommendations, summarizing findings and future work.

CHAPTER 2

LITERATURE REVIEW

The evolution of Large Language Models (LLMs) has transcended the boundaries of simple linguistic synthesis, moving toward the realization of agentic systems capable of autonomous, multi-step task execution (Jian, 2025). While the generative prowess of these models is undeniable, their deployment as singular autonomous entities has exposed significant intrinsic limitations, most notably the propensity for hallucination and the constraints of their autoregressive nature (Tran *et al.*, 2025). These models often struggle with "slow-thinking" processes (Hagendorff *et al.*, 2023) the systematic, deliberative reasoning required for complex problem-solving and face diminishing returns as scaling laws meet the reality of open-ended computational logic (Kaplan *et al.*, 2020). To bridge these gaps, most implementations of agentic applications have pivoted toward Multi-Agent Systems (MAS), under the observation that a collective of specialized agents consistently outperforms individual models by distributing cognitive load and introducing cross-verification (Albada, 2025a). This shift, however, necessitates a rigorous definition of how these agents collaborate, communicate, and are orchestrated to ensure that the collective output remains coherent and goal-oriented.

Tran *et al.* (2025) describes orchestration in the context of MAS as a multifaceted discipline encompassing collaboration types, coordination strategies, and specific communication structures designed to manage the flow of information between disparate agents. A primary focus within this field is the "sequencing" of agent interventions, which dictates the operational logic of the workflow. In many current frameworks, this sequencing is either statically defined through manual scripting or relies on heuristic-based "context hand-offs," where the state of a task is passed linearly from one agent to the next based on probabilistic triggers. Such strategies, while functional for experimental or simple workflows, frequently fail to meet the reliability benchmarks required of enterprise systems, where reliability is defined by the consistency and accuracy of generated outputs (Menshawy and Fahmy, 2025). Because enterprise deployments demand that reliability be maintained through rigorous testing, validation phases, and robust version control, the inherent "stochastic guessing" of probabilistic sequencing introduces unacceptable inaccuracies. The reliance on models that may be compromised by training biases or learning anomalies necessitates a move toward

orchestration that does not just "predict" the next step, but validates it against a quality assurance framework ensuring that every agentic transition is as dependable as a traditional enterprise software deployment.

To anchor these agentic decisions in reality, researchers have historically looked to Knowledge Graphs (KGs) as a robust mechanism for grounding LLM outputs (Kau *et al.*, 2024). Knowledge Graphs provide a structured, symbolic representation of a domain, offering a "Source of Truth" and a relational depth that probabilistic models inherently lack (Hogan *et al.*, 2021). By integrating KGs into the decision-making loop, systems can verify agent claims against a persistent, factual schema, thereby mitigating the risks of stochastic drift and ensuring semantic consistency. The intersection of graph-based reasoning and agent orchestration represents a significant frontier in AI research, as it promises to transform the way agents understand their environment and their own functional capabilities. This historical precedent for KG-driven grounding suggests that the next logical step in the evolution of MAS is the use of graph structures not just for static data retrieval, but as the fundamental logic gate for orchestration itself.

This chapter provides a comprehensive examination of existing approaches to multi-agent orchestration, tracing the trajectory from general collaboration frameworks to specialized strategies focused on agent sequencing and context hand-off, as well as emerging approaches that propose dynamic methods for managing these transitions. By critically analyzing these methodologies, the review highlights a persistent limitation: the lack of a truly dynamic and deterministic way to decide the order of execution in complex, non-linear workflows. Many current "dynamic" models still operate within the vacuum of the LLM's context window, leaving the sequencing vulnerable to the same autoregressive failures the MAS architecture was intended to solve. Through this analysis, we establish the critical need for a fact-grounded approach to dynamic agent sequencing. This leads directly to the core contribution of this project: a Knowledge Graph-Based Multi-Agent Orchestration Framework that utilizes a deterministic world model to ensure that every agentic transition is logically validated and anchored in the objective reality of the graph.

2.1 Introduction to Concepts

This section provides comprehensive explanations of the key concepts from LLM-Based Agents, Multi-Agent Systems to Knowledge Graphs, ensuring a solid theoretical foundation.

2.1.1 LLM-Based Agents

To define the foundational components of the framework, it is first necessary to establish the architectural anatomy of an LLM-Based Agent. Unlike a standard language model, which functions as a passive text generator, an agent is a semi-autonomous system that uses the LLM as its "reasoning engine" to interact with an external environment. The literature Xi *et al.* (2023) generally decomposes an agent into three primary pillars Memory, Planning, and Tool Use often abstracted as the "brain," "perception," and "action" of the system.

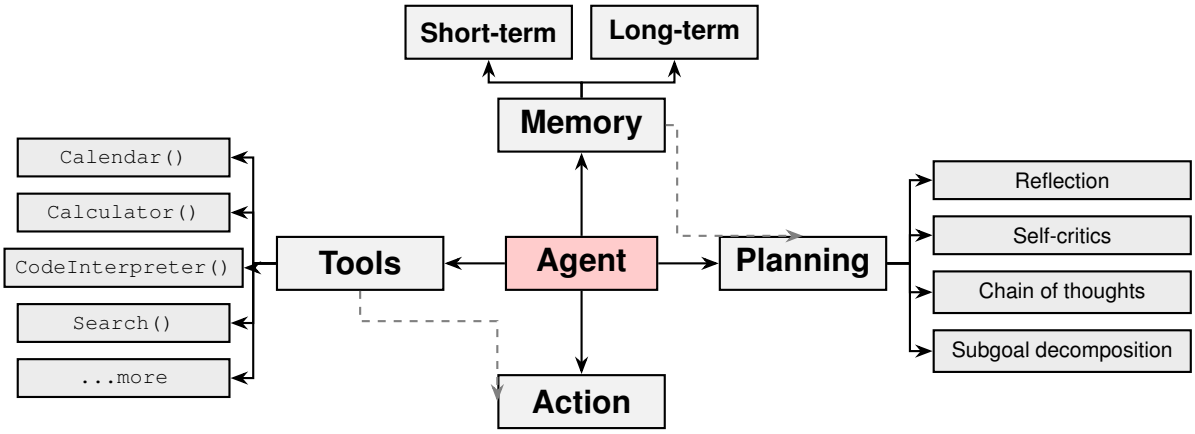

Figure 2.1 The modular architecture of an LLM-based Agent, illustrating the interaction between reasoning core, memory, and tools.

Memory allows the agent to maintain state over time, categorizing information into short-term storage (the context window) and long-term storage (external databases or Retrieval-Augmented Generation) (Albada, 2025b).

Tool use (Schick *et al.*, 2023; Parisi *et al.*, 2022) enables the agent to transcend the "digital silo" of its training data by invoking external APIs, executing code, or querying databases to retrieve real-time information.

However, the most critical and most volatile component is the Planning layer, which governs how the agent cognitively breaks down a complex objective into executable steps. The current state-of-the-art in agentic planning relies almost exclusively on prompt-based techniques

designed to elicit "System 2" thinking deliberative, analytical reasoning from the model. Strategies such as Chain-of-Thought (CoT) force the model to generate intermediate reasoning steps (Wei *et al.*, 2023), while Tree-of-Thoughts (ToT) allows the agent to explore multiple reasoning branches and self-evaluate their progress in a non-linear fashion (Yao *et al.*, 2023a). More advanced frameworks like ReAct (Reasoning and Acting) create a feedback loop where the model interleaves thoughts, actions, and observations to refine its trajectory (Yao *et al.*, 2023b).

While these methods significantly improve performance on isolated tasks, they remain purely probabilistic and heuristic. The planning is "soft-coded" within the prompt, meaning the agent's logic is only as reliable as the model's next-token prediction. There are no hard constraints to prevent the agent from skipping a vital step or proposing a logically impossible action, as the "plan" is essentially a sophisticated guess based on linguistic patterns rather than a validated structural map.

In terms of Task Execution, LLM-based agents typically operate through either direct handling or tool invocation. In the former, the LLM processes information and generates a response based on its internal weights; in the latter, the agent must translate its intent into a structured output (Liu *et al.*, 2024), such as a JSON-formatted function call, to interact with an execution tool. The tool returns a raw response, which the LLM must then re-process into a natural language summary or use as a catalyst for the next step in the plan. This execution phase is where the "grounding" of the agent is tested, as it requires the model to precisely map its internal reasoning to the strict syntactic requirements of external software. Any misalignment between the LLM's generative output and the tool's expected schema results in execution failure, highlighting the fragility of relying on linguistic intuition for technical interoperability.

Despite the sophistication of these components, single-agent systems frequently succumb to catastrophic failure when applied to complex, multi-stage enterprise workflows. These failures are primarily attributed to the "unconstrained" nature of prompt-based planning (Rana and Kumar, 2025). Without an external governing structure, a single agent often enters a hallucination recursive loop, where it attempts to correct an initial error by generating further inaccurate logic based on that false premise (Latitude, 2024; Cheng *et al.*, 2025). Additionally, as the task complexity increases, the agent suffers from contextual overload; the accumulation of previous thoughts, tool outputs, and observations clutters the context window,

leading to a precipitous degradation in reasoning quality (Anthropic, 2024). This architectural fragility underscores the necessity for moving away from isolated, prompt-driven agents toward a more rigid, orchestrated multi-agent environment where the "world model" is held by a Knowledge Graph rather than the volatile and finite memory of a single model.

2.1.2 Multi-Agent Systems

The evolution of agentic architectures from single-model entities to Multi-Agent Systems (MAS) represents a strategic transition toward functional decomposition and collaborative problem-solving. In a MAS environment, a complex global objective is partitioned into discrete sub-tasks, each managed by specialized agents possessing distinct roles, personas, and toolsets (Yang *et al.*, 2024). This shift is predicated on the observation that the collective intelligence of a modular system consistently outperforms the reasoning capacity of a monolithic LLM, particularly in tasks requiring high cognitive diversity or parallel execution (Li *et al.*, 2024; Guo *et al.*, 2024). By isolating specific domains of expertise such as an agent dedicated solely to code generation and another to quality assurance the system introduces a layer of intersubjective verifiability, where agents can critique and refine each other's outputs, effectively serving as a circuit breaker for the "hallucination recursive loops" prevalent in single-model configurations.

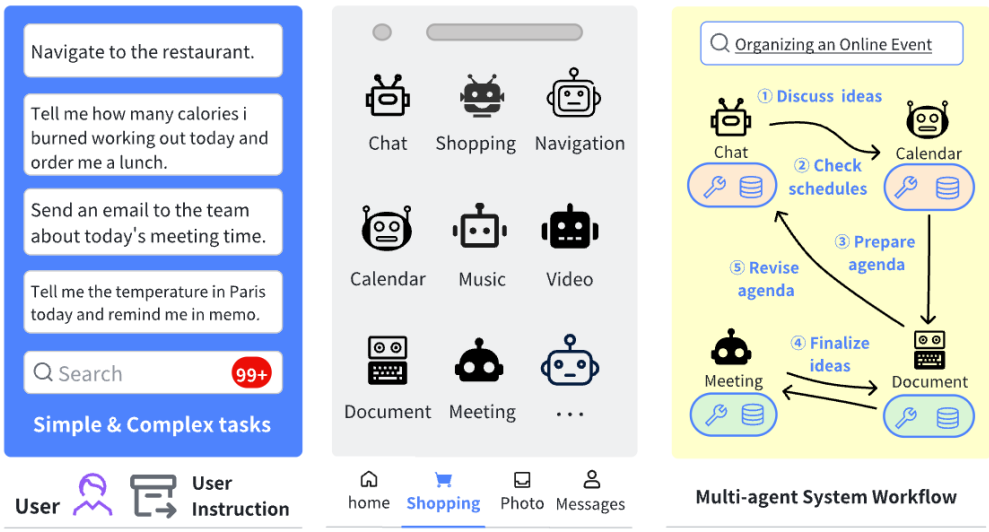

Figure 2.2 Illustration of a Multi-Agent System (MAS) architecture, reproduced from Yang *et al.* (2024)

Despite these advantages, the transition to MAS introduces a significant coordination overhead, commonly referred to as the "Coordination Tax" (Kim *et al.*, 2025), as the number

of agents increases, the complexity of managing their interactions grows exponentially, necessitating sophisticated communication protocols and collaboration strategies. Wang *et al.* (2025) distinguishes between "Horizontal" collaboration, where agents interact as peers in an unstructured manner, and "Vertical" or "Hierarchical" orchestration, where a lead agent or central controller dictates the workflow. Without a formal governing structure, even these collaborative models remain bound by the limitations of the underlying models' ability to maintain a consistent state across fragmented context windows.

Consequently, Valmeekam *et al.* (2023) underscores that while Large Language Models (LLMs) can generate text that mimics the structure of a plan, they consistently fail at logically validated execution. This empirical "planning gap" demonstrates that agent collaboration and orchestration cannot be left to the probabilistic nature of the LLM alone. By shifting the responsibility of workflow logic from the LLM to a structured world model, the framework ensures that every agentic action is not just linguistically plausible, but mathematically verified and fully auditable. This transition effectively moves the system from a paradigm of "stochastic sequencing" to one of "deterministic coordination," providing the reliability required for enterprise-grade applications.

CHAPTER 3

CHAPTER THREE TITLE GOES HERE

3.1 FIRST SECTION TITLE GOES HERE

3.1.1 Your subsection

CHAPTER 4

CHAPTER TITLE HERE

4.1 FIRST SECTION TITLE GOES HERE

4.1.1 Your subsection

CHAPTER 5

CONCLUSION AND RECOMMENDATION

5.1 Conclusion

5.2 Recommendations

REFERENCES

Albada, M. (2025a). "Title of Chapter". In: *Building Applications with AI Agents*. O'Reilly Media, p. 163.

— (2025b). "Title of Chapter". In: *Building Applications with AI Agents*. O'Reilly Media, pp. 116 –123.

Amayuelas, A., Sain, J., Kaur, S., and Smiley, C. (2025). *Grounding LLM Reasoning with Knowledge Graphs*. arXiv: 2502.13247 [cs.CL]. URL: https://arxiv.org/abs/2502.13247.

Anthropic (2024). *Effective Context Engineering for AI Agents*. Accessed: 2026-03-31. URL: https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

Cheng, J., Su, T., Yuan, J., He, G., Liu, J., Tao, X., Xie, J., and Li, H. (2025). *Chain-of-Thought Prompting Obscures Hallucination Cues in Large Language Models: An Empirical Evaluation*. arXiv: 2506.17088 [cs.CL]. URL: https://arxiv.org/abs/2506.17088.

Derouiche, H., Brahmi, Z., and Mazeni, H. (2025). *Agentic AI Frameworks: Architectures, Protocols, and Design Challenges*. arXiv: 2508.10146 [cs.AI]. URL: https://arxiv.org/abs/2508.10146.

Elementum Team (2026). "Deterministic vs. Probabilistic AI: The Enterprise Guide to Building Workflows That Scale". In: *Elementum AI Blog*. Accessed: 2026-03-26. URL: https://www.elementum.ai/blog/deterministic-vs-probabilistic-ai.

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., and Zhang, X. (2024). *Large Language Model based Multi-Agents: A Survey of Progress and Challenges*. arXiv: 2402.01680 [cs.CL]. URL: https://arxiv.org/abs/2402.01680.

Hagendorff, T., Fabi, S., and Kosinski, M. (2023). "Human-like intuitive behavior and reasoning biases emerged in large language models but disappeared in ChatGPT". In: *Nature Computational Science* 3.10, pp. 833–838. DOI: 10.1038/s43588-023-00529-1.

Hogan, A., Blomqvist, E., Cochez, M., D'amato, C., Melo, G. D., Gutierrez, C., Kirrane, S., Gayo, J. E. L., Navigli, R., Neumaier, S., Ngomo, A.-C. N., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., and Zimmermann, A. (July 2021). "Knowledge Graphs". In: *ACM Computing Surveys* 54.4, 1–37. ISSN: 1557-7341. DOI: 10.1145/3447772. URL: http://dx.doi.org/10.1145/3447772.

Jian, Y. (2025). "Conceptual Foundations of LLM-Powered Agents: From Language Processing to Autonomous Reasoning". In: *Scientific Journal of Intelligent Systems Research* 7.4, 29–35. DOI: 10.54691/atmxvb43. URL: http://dx.doi.org/10.54691/atmxvb43.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., Amodei, D., *et al.* (2020). "Scaling Laws for Neural Language Models". In: *arXiv:2001.08361*. arXiv: 2001.08361 [cs.LG].

Kau, A., He, X., Nambissan, A., Astudillo, A., Yin, H., and Aryani, A. (2024). *Combining Knowledge Graphs and Large Language Models*. arXiv: 2407.06564 [cs.CL]. URL: https://arxiv.org/abs/2407.06564.

Kim, Y., Gu, K., Park, C., Park, C., Schmidgall, S., Heydari, A. A., Yan, Y., Zhang, Z., Zhuang, Y., Malhotra, M., Liang, P. P., Park, H. W., Yang, Y., Xu, X., Du, Y., Patel, S., Althoff, T., McDuff, D., and Liu, X. (2025). *Towards a Science of Scaling Agent Systems*. arXiv: 2512.08296 [cs.AI]. URL: https://arxiv.org/abs/2512.08296.

LangChain Inc. (2024). *LangGraph Graph API*. https://docs.langchain.com/oss/python/langgraph/graph-api. Accessed: 2026-03-26.

Latitude (2024). *How Task Complexity Drives Error Propagation in LLMs*. Accessed: 2026-03-31. URL: https://latitude.so/blog/how-task-complexity-drives-error-propagation-in-llms.

Li, J., Zhang, Q., Yu, Y., Fu, Q., and Ye, D. (2024). "More Agents Is All You Need". In: *Transactions on Machine Learning Research*. URL: https://openreview.net/forum?id=bgzUSZ8aeg.

Liu, M. X., Liu, F., Fiannaca, A. J., Koo, T., Dixon, L., Terry, M., and Cai, C. J. (May 2024). ""We Need Structured Output": Towards User-centered Constraints on Large Language Model Output". In: *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. CHI '24. ACM, 1–9. DOI: 10.1145/3613905.3650756. URL: http://dx.doi.org/10.1145/3613905.3650756.

Menshawy, A. and Fahmy, M. (2025). *LLMs in Enterprise: Design Strategies for Large Language Model Development, Design Patterns and Best Practices*. Packt Publishing, p. 71. ISBN: 978-1836203070.

Nath, A. (2025). "A Practical Perspective on Orchestrating AI Agent Systems with DAGs". In: *Medium*. Accessed: 2026-03-26. URL: https://medium.com/@arpitnath42/a-practical-perspective-on-orchestrating-ai-agent-systems-with-dags-c9264bf38884.

Nizar, F., Lumer, E., Gulati, A., Basavaraju, P. H., and Subbiah, V. K. (2025). *Agent-as-a-Graph: Knowledge Graph-Based Tool and Agent Retrieval for LLM Multi-Agent Systems*. arXiv: 2511.18194 [cs.CL]. URL: https://arxiv.org/abs/2511.18194.

Parisi, A., Zhao, Y., and Fiedel, N. (2022). *TALM: Tool Augmented Language Models*. arXiv: 2205.12255 [cs.CL]. URL: https://arxiv.org/abs/2205.12255.

Rana, A. and Kumar, G. (2025). *Model-First Reasoning LLM Agents: Reducing Hallucinations through Explicit Problem Modeling*. arXiv: 2512.14474 [cs.AI]. URL: https://arxiv.org/abs/2512.14474.

Sanmartin, D. (2024). *KG-RAG: Bridging the Gap Between Knowledge and Creativity*. arXiv: 2405.12035 [cs.AI]. URL: https://arxiv.org/abs/2405.12035.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. arXiv: 2302.04761 [cs.CL]. URL: https://arxiv.org/abs/2302.04761.

Shankar, S. (2025). *LangGraph as a DAG: Rethinking Data Pipeline Orchestration*. Medium article. URL: https://medium.com/@srikrishnan.tech/langgraph-as-a-dag-rethinking-data-pipeline-orchestration-f089ccea175b.

Tran, K.-T., Dao, D., Nguyen, M.-D., Pham, Q.-V., O'Sullivan, B., and Nguyen, H. D. (2025). *Multi-Agent Collaboration Mechanisms: A Survey of LLMs*. arXiv: 2501.06322 [cs.AI]. URL: https://arxiv.org/abs/2501.06322.

Valmeekam, K., Marquez, M., Olmo, A., Sreedharan, S., and Kambhampati, S. (2023). *PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change*. arXiv: 2206.10498 [cs.CL]. URL: https://arxiv.org/abs/2206.10498.

Wang, Y., Pan, Y., Su, Z., Deng, Y., Zhao, Q., Du, L., Luan, T. H., Kang, J., and Niyato, D. (2025). *Large Model Based Agents: State-of-the-Art, Cooperation Paradigms, Security and Privacy, and Future Trends*. arXiv: 2409.14457 [cs.AI]. URL: https://arxiv.org/abs/2409.14457.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2023). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. arXiv: 2201.11903 [cs.CL]. URL: https://arxiv.org/abs/2201.11903.

Weng, L. (2023). "LLM-powered Autonomous Agents". In: *Lil'Log*. Accessed: 2026-03-26. URL: https://lilianweng.github.io/posts/2023-06-23-agent/.

Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., Zheng, R., Fan, X., Wang, X., Xiong, L., Zhou, Y., Wang, W., Jiang, C., Zou, Y., Liu, X., Yin, Z., Dou, S., Weng, R., Cheng, W., Zhang, Q., Qin, W., Zheng, Y., Qiu, X., Huang, X., and Gui, T. (2023). *The Rise and Potential of Large Language Model Based Agents: A Survey*. arXiv: 2309.07864 [cs.AI]. URL: https://arxiv.org/abs/2309.07864.

Yang, Y., Peng, Q., Wang, J., Wen, Y., and Zhang, W. (2024). *LLM-based Multi-Agent Systems: Techniques and Business Perspectives*. arXiv: 2411.14033 [cs.AI]. URL: https://arxiv.org/abs/2411.14033.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., and Narasimhan, K. (2023a). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. arXiv: 2305.10601 [cs.CL]. URL: https://arxiv.org/abs/2305.10601.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023b). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv: 2210.03629 [cs.CL]. URL: https://arxiv.org/abs/2210.03629.